

Mapping ER Model to a logical schema

Entity-Relationship (E-R) diagram to a logical data model, you need to perform the following steps:

Identify entities and relationships: Start by identifying entities and relationships in the E-R diagram. An entity is a real-world object, person, or concept that can be represented in a database, while a relationship represents the connection between two entities.

Define attributes for entities: Next, define attributes for each entity that describe the characteristics of that entity. These attributes will become columns in the tables representing the entities in the logical model.

Create tables for entities: Create tables for each entity, and include columns for each of the attributes identified in the previous step.

Define relationships between tables: Define relationships between tables by adding foreign keys to the appropriate tables. A foreign key is a column in one table that refers to a primary key in another table.

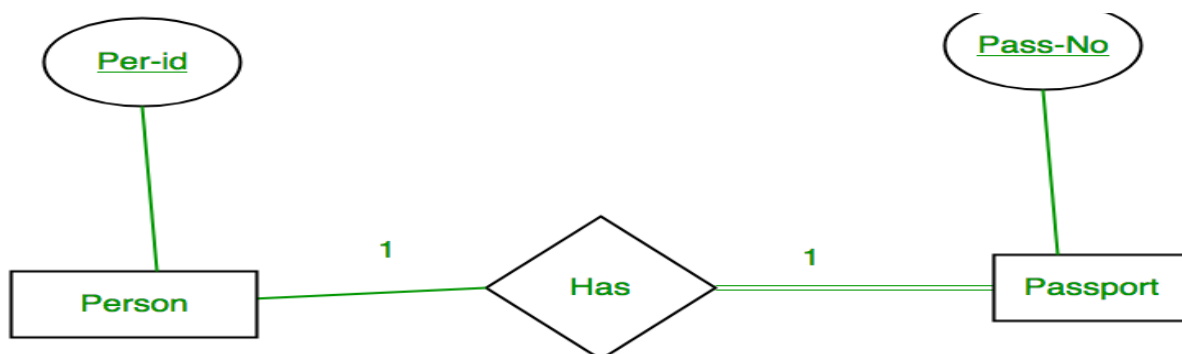
Normalize the data: Normalize the data to minimize data redundancy and improve data integrity. This process involves breaking down tables into smaller, more specialized tables to eliminate repeating data and ensure data consistency.

Add constraints: Add constraints to the data model to enforce business rules and ensure data integrity. Constraints can include primary keys, unique constraints, and referential integrity constraints.

Map the relationships: Map the relationships between tables using appropriate data relationships, such as one-to-one, one-to-many, or many-to-many relationships.

Let's understand how to map the relation with the ER model for different scenarios:

Case 1: Binary Relationship with 1:1 cardinality with total participation of an entity



A person has 0 or 1 passport number and Passport is always owned by 1 person. So it is 1:1 cardinality with full participation constraint from Passport.

First Convert each entity and relationship to tables. Person table corresponds to Person Entity with key as Per-Id. Similarly the Passport table corresponds to Passport Entity with key as Pass-No. Has Table represents relationship between Person and Passport (Which person has which passport). So it will take attribute Per-Id from Person and Pass-No from Passport.

Person		Has		Passport	
<u>Per-Id</u>	Other Person Attribute	<u>Per-Id</u>	Pass-No	<u>Pass-No</u>	Other PassportAttribute
PR1	–	PR1	PS1	PS1	–
PR2	–	PR2	PS2	PS2	–
PR3	–				

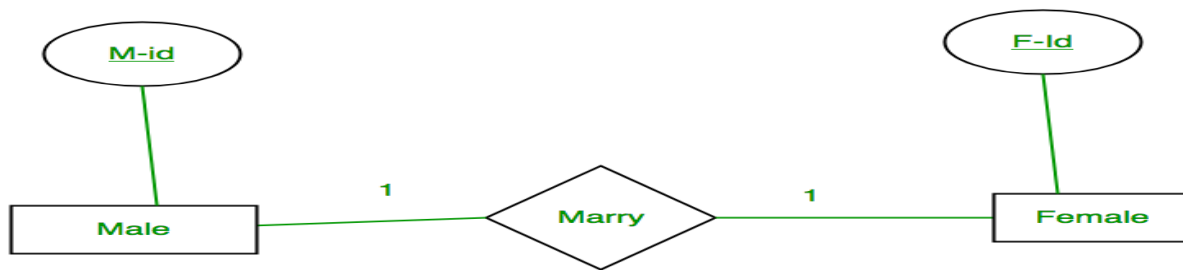
Table 1

As we can see from Table 1, each Per-Id and Pass-No has only one entry in Has Table. So we can merge all three tables into 1 with attributes shown in Table 2. Each Per-Id will be unique and not null. So it will be the key. Pass-No can't be key because for some person, it can be NULL.

<u>Per-Id</u>	Other Person Attribute	Pass-No	Other PassportAttribute
---------------	------------------------	---------	-------------------------

Table 2

Case 2: Binary Relationship with 1:1 cardinality and partial participation of both entities



A male marries 0 or 1 female and vice versa as well. So it is 1:1 cardinality with partial participation constraint from both. First Convert each entity and relationship to tables. Male table corresponds to Male Entity with key as M-Id. Similarly Female table corresponds to Female Entity with key as F-Id. Marry Table represents relationship between Male and Female (Which Male marries which female). So it will take attribute M-Id from Male and F-Id from Female.

Male		Marry		Female	
M-Id	Other Male Attribute	M-Id	F-Id	F-Id	Other FemaleAttribute
M1	–	M1	F2	F1	–
M2	–	M2	F1	F2	–
M3	–			F3	–

Table 3

As we can see from Table 3, some males and some females do not marry. If we merge 3 tables into 1, for some M-Id, F-Id will be NULL. So there is no attribute which is always not NULL. So we can't merge all three tables into 1. We can convert into 2 tables. In table 4, M-Id who are married will have F-Id associated.

For others, it will be NULL. Table 5 will have information of all females. Primary Keys have been underlined.

<u>M-I</u> d	Other Male Attribute	F-I d
-----------------	-------------------------	----------

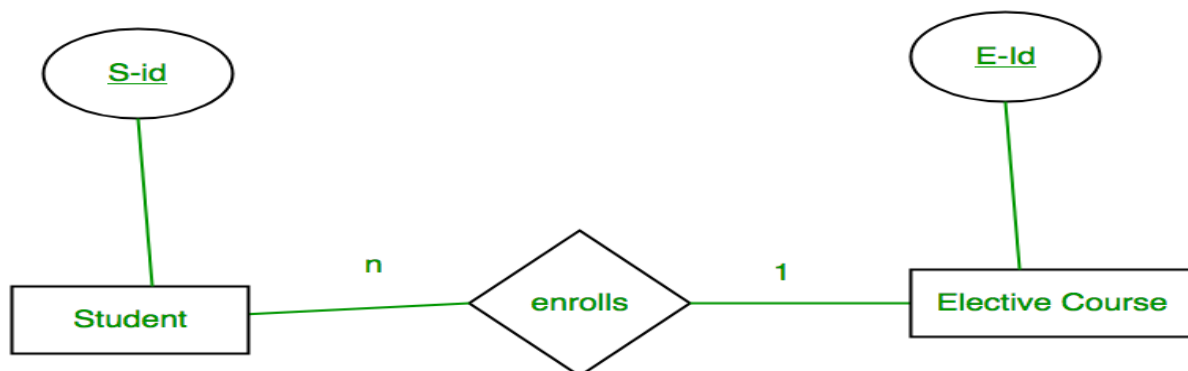
Table 4

F-I d	Other FemaleAttribute
----------	--------------------------

Table 5

Note: Binary relationship with 1:1 cardinality will have 2 table if partial participation of both entities in the relationship. If atleast 1 entity has total participation, number of tables required will be 1.

Case 3: Binary Relationship with n: 1 cardinality



In this scenario, every student can enroll only in one elective course but for an elective course there can be more than one student. First Convert each entity and relationship to tables. Student table corresponds to Student Entity with key as S-Id. Similarly Elective_Course table corresponds to Elective_Course Entity with key as E-Id. Enrolls Table represents relationship between Student and

Elective_Course (Which student enrolls in which course). So it will take attribute S-Id from Student and E-Id from Elective_Course.

Student		Enrolls		Elective_Course	
S-Id	Other Student Attribute	S-Id	E-Id	E-Id	Other Elective CourseAttribute
S1	–	S1	E1	E1	–
S2	–	S2	E2	E2	–
S3	–	S3	E1	E3	–
S4	–	S4	E1		

Table 6

As we can see from Table 6, S-Id is not repeating in Enrolls Table. So it can be considered as a key of Enrolls table. Both Student and Enrolls Table's key is same; we can merge it as a single table. The resultant tables are shown in Table 7 and Table 8. Primary Keys have been underlined.

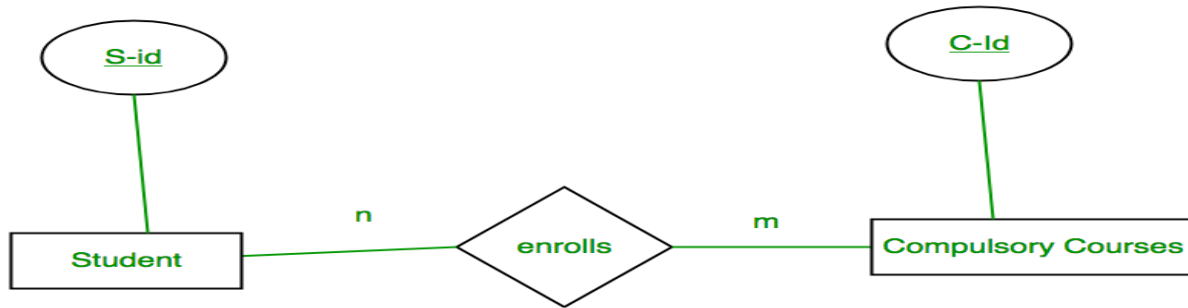
<u>S-Id</u>	Other Student Attribute	<u>E-Id</u>
-------------	-------------------------	-------------

Table 7

<u>E-Id</u>	Other Elective CourseAttribute
-------------	--------------------------------

Table 8

Case 4: Binary Relationship with m: n cardinality



In this scenario, every student can enroll in more than 1 compulsory course and for a compulsory course there can be more than 1 student. First Convert each entity and relationship to tables. Student table corresponds to Student Entity with key as S-Id. Similarly Compulsory_Courses table corresponds to Compulsory Courses Entity with key as C-Id. Enrolls Table represents relationship between Student and Compulsory_Courses (Which student enrolls in which course). So it will take attribute S-Id from Person and C-Id from Compulsory_Courses.

Student		Enrolls		Compulsory_Courses	
<u>S-Id</u>	Other Student Attribute	<u>S-Id</u>	<u>C-Id</u>	<u>C-Id</u>	Other Compulsory CourseAttribute
S1	–	S1	C1	C1	–
S2	–	S1	C2	C2	–
S3	–	S3	C1	C3	–
S4	–	S4	C3	C4	–
		S4	C2		
		S3	C3		

Table 9

As we can see from Table 9, S-Id and C-Id both are repeating in Enrolls Table. But its combination is unique; so it can be considered as a key of Enrolls table. All tables' keys are different, these can't be merged. Primary Keys of all tables have been underlined.

Mapping EER Model to a logical schema

1. To draw EER diagrams first, we have to identify all entities.
 - After we found entities from the scenario you should check whether those entities have sub-entities. If so you have to mark sub-entities in your diagram.
 - Dividing entities into sub-entities we called as specialization. And combining sub-entities to one entity is called a generalization.

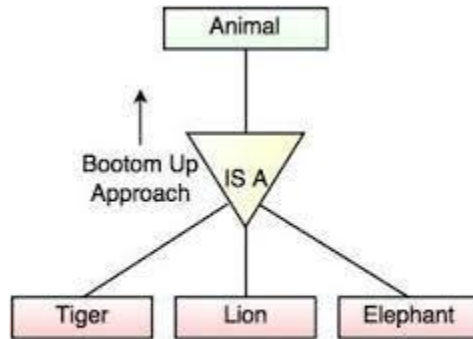


Fig. Generalization

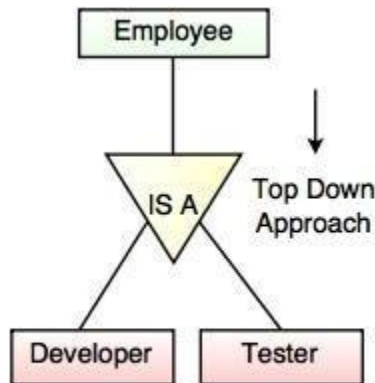
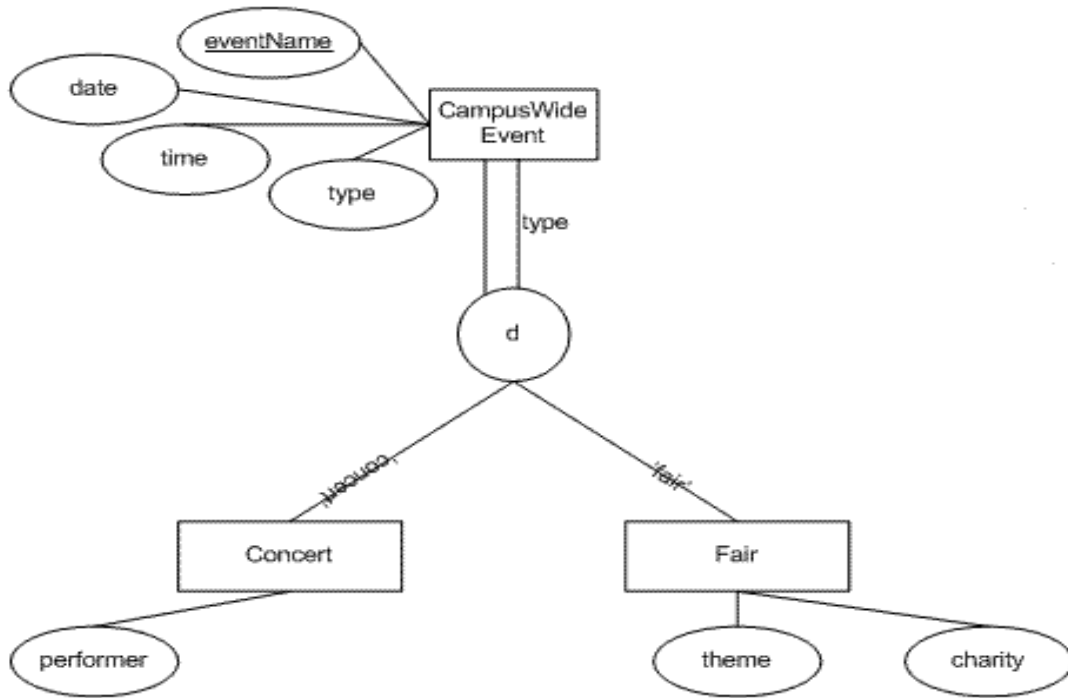


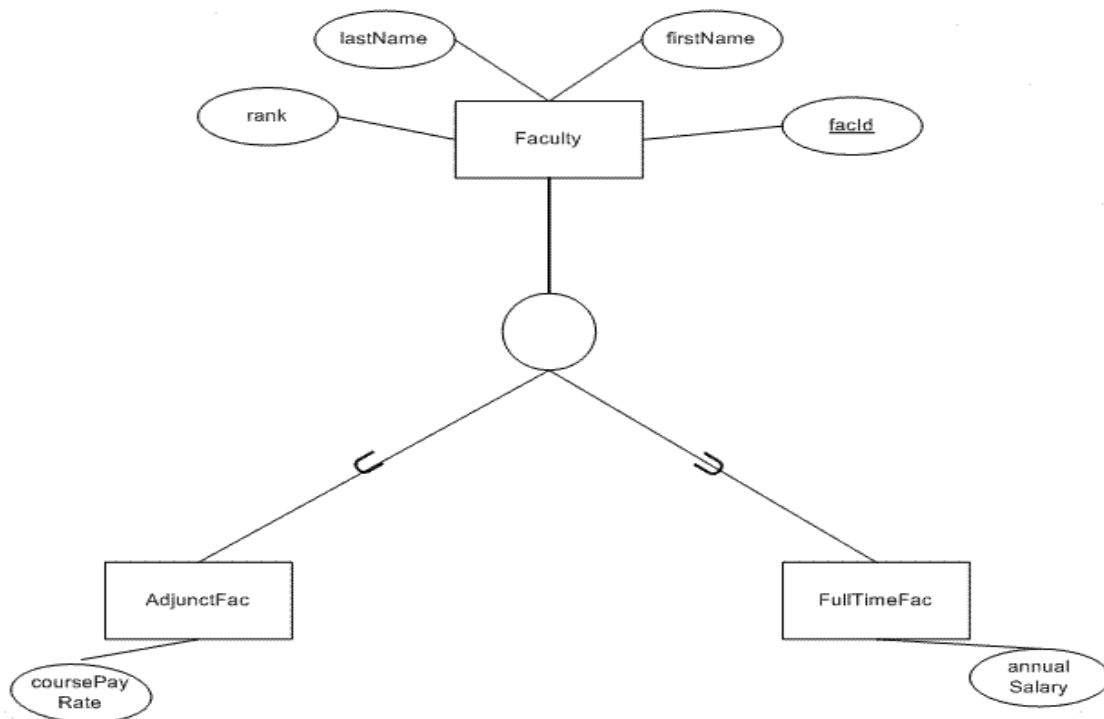
Fig. Specialization

2. Then you have to identify relationships between entities and mark them.
 3. Add attributes for entities. Give meaningful attribute names so they can be understood easily.
 4. Mark the cardinalities and participation
- It is also required to check whether sub-entities totally depend on the main entity or not. And you should mark it.
- If all members in the superclass(main entity) participate in either one subclass it is known as total participation. It is marked by double lines.



Total Participation

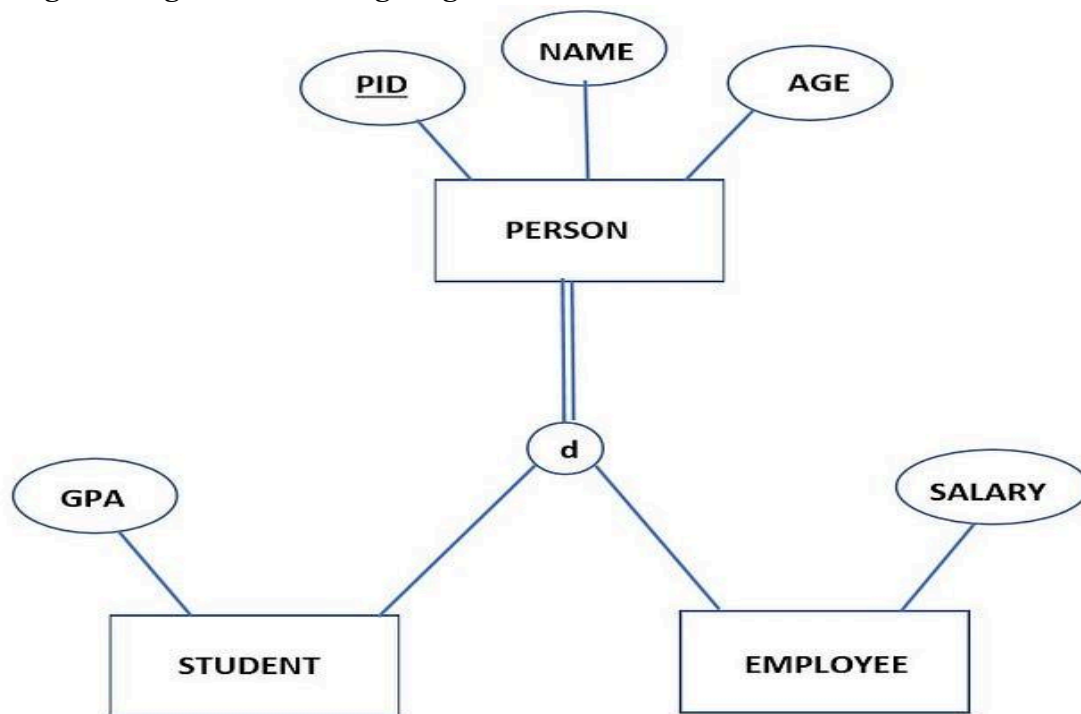
If at least one member in the superclass does not participate in subclass it is known as partial participation. It is denoted by one single line.



Partial Participation

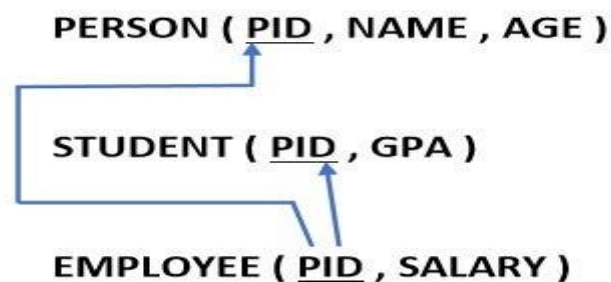
EER diagrams into relational schemas.

Let us go through the following diagram.



There are four ways to draw relational schema for an EER. You have to choose the most suitable one. In the end, I'll give you the guidelines on how to choose the best and most suitable way.

First method



Here we write separate relations to all superclass entities and subclass entities. And here we have to write the superclass entities' primary key to all subclass entities and then map them as shown above. Note that we write only the attributes belongs to each entity.

Second method

STUDENT (PID , GPA , NAME , AGE)

EMPLOYEE (PID , SALARY , NAME , AGE)

Here we do not write the superclass entity but in each subclass entity, we write all attributes that are in superclass entity.

Third method

PERSON (PID , NAME , AGE , SALARY , GPA , PERSONTYPE)

Here we write only the superclass entity and write all the attributes which belong to subclass entities. Specialty in here is that to identify that PERSON is an EMPLOYEE or STUDENT we add a column as PERSONTYPE. After the table creates we can mark as a STUDENT or EMPLOYEE.

Fourth method

PERSON (PID , NAME , AGE , SALARY , GPA , STUDENT , EMPLOYEE)

Here instead of PERSONTYPE, we write STUDENT and EMPLOYEE both.

Mapping of higher degree relationships

Degree of Relationship

In DBMS, a degree of relationship represents the number of entity types that are associated with a relationship. For example, we have two entities, one is a student and the other is a bag and they are connected with the primary key and foreign key. So, here we can see that the degree of relationship is 2 as 2 entities are associating in a relationship.

Types of degree

Now, based on the number of linked entity types, we have 4 types of degrees of relationships.

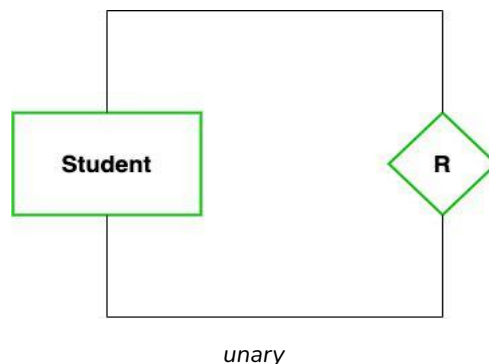
1. Unary
2. Binary
3. Ternary
4. N-ary

Let's discuss them one by one with the help of examples.

Unary (Degree 1)

In this type of relationship, both the associating entity types are the same. So, we can say that unary relationships exist when both entity types are the same and we call them the degree of relationship is 1. In other words, in a relation only one entity set is participating then such type of relationship is known as a unary relationship.

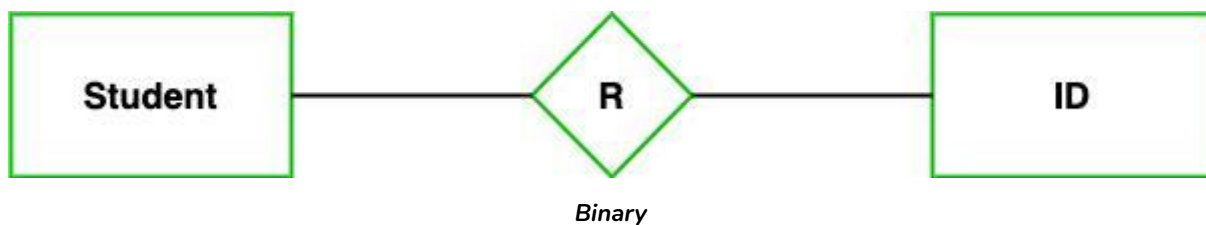
Example: In a particular class, we have many students, there are monitors too. So, here class monitors are also students. Thus, we can say that only students are participating here. So the degree of such type of relationship is 1.



Binary (Degree 2)

In a Binary relationship, there are two types of entity associates. So, we can say that a Binary relationship exists when there are two types of entity and we call them a degree of relationship is 2. Or in other words, in a relation when two entity sets are participating then such type of relationship is known as a binary relationship. This is the most used relationship and one can easily be converted into a relational table.

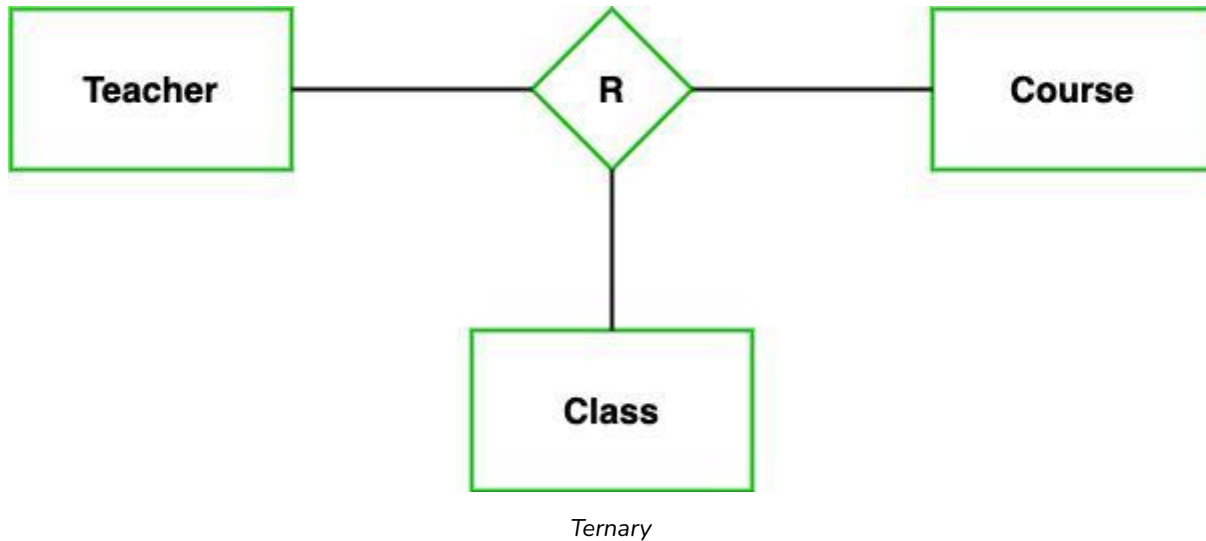
Example: We have two entity types 'Student' and 'ID' where each 'Student' has his 'ID'. So, here two entity types are associating we can say it is a binary relationship. Also, one 'Father' can have many 'daughters' but each 'daughter' should belong to only one 'father'. We can say that it is a one-to-many binary relationship.



Ternary (Degree 3)

In the Ternary relationship, there are three types of entity associates. So, we can say that a Ternary relationship exists when there are three types of entity and we call them a degree of relationship is 3. Since the number of entities increases due to this, it becomes very complex to turn E-R into a relational table. Now let's understand with the examples.

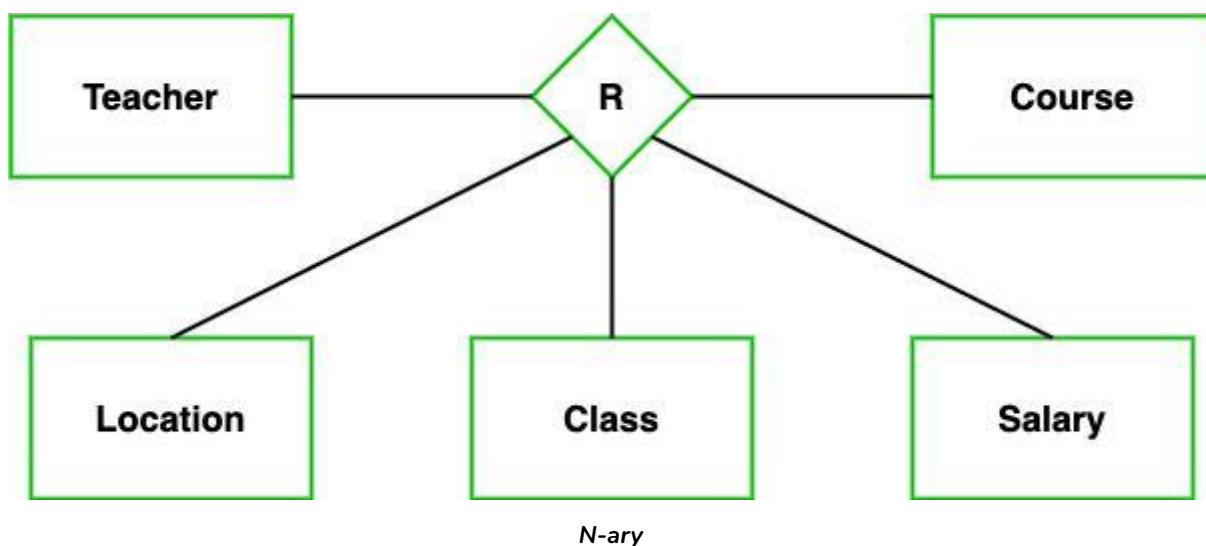
Example: We have three entity types 'Teacher', 'Course', and 'Class'. The relationship between these entities is defined as the teacher teaching a particular course, also the teacher teaches a particular class. So, here three entity types are associating we can say it is a ternary relationship.



N-ary (n Degree)

In the N-ary relationship, there are n types of entity that associates. So, we can say that an N-ary relationship exists when there are n types of entities. There is one limitation of the N-ary relationship, as there are many entities so it is very hard to convert into an entity, relational table. So, this is very uncommon, unlike binary which is very much popular.

Example: We have 5 entities Teacher, Class, Location, Salary, Course. So, here five entity types are associating we can say an n-ary relationship is 5.



Relationship of higher degree

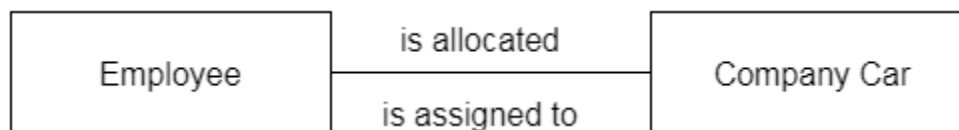
The degree of relationship can be defined as the number of occurrences in one entity that is associated with the number of occurrences in another entity.

There is the three degree of relationship:

1. One-to-one (1:1)
2. One-to-many (1:M)
3. Many-to-many (M:N)

1. One-to-one

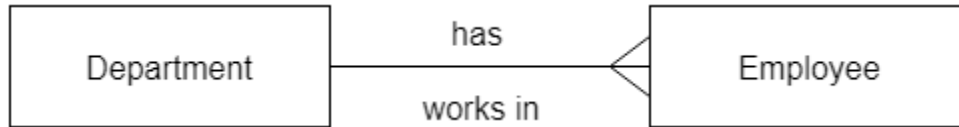
- In a one-to-one relationship, one occurrence of an entity relates to only one occurrence in another entity.
- A one-to-one relationship rarely exists in practice.
- For example: if an employee is allocated a company car then that car can only be driven by that employee.
- Therefore, employee and company car have a one-to-one relationship.



2. One-to-many

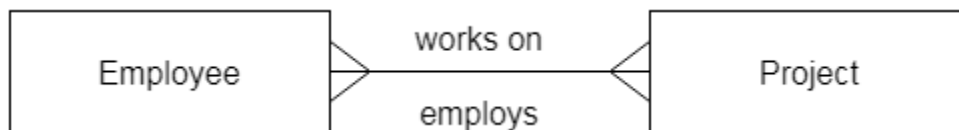
- In a one-to-many relationship, one occurrence in an entity relates to many occurrences in another entity.
- For example: An employee works in one department, but a department has many employees.

- Therefore, department and employee have a one-to-many relationship.



3. Many-to-many

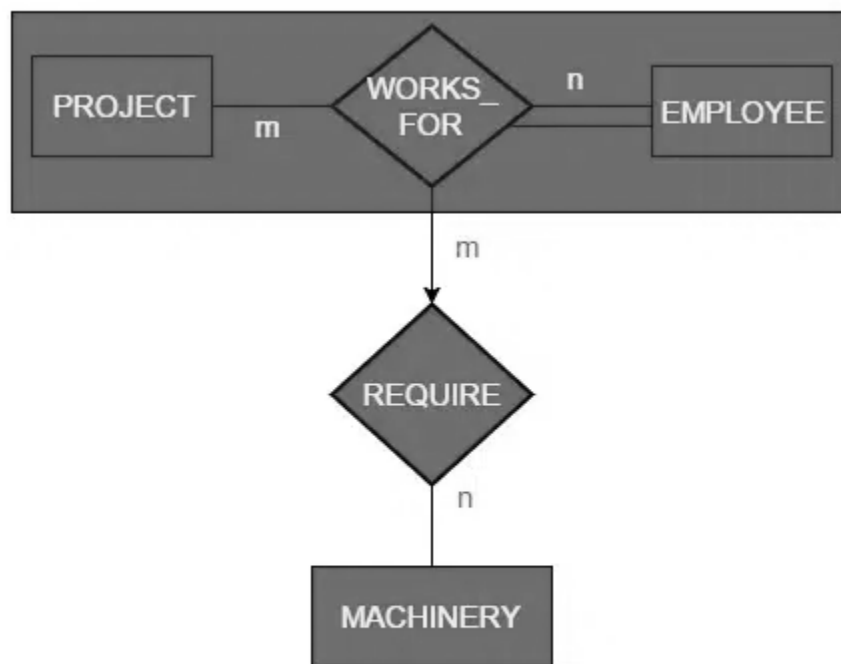
- In a many-to-many relationship, many occurrences in an entity relate to many occurrences in another entity.
- Same as a one-to-one relationship, the many-to-many relationship rarely exists in practice.
- For example: At the same time, an employee can work on several projects, and a project has a team of many employees.
- Therefore, employee and project have a many-to-many relationship.



Aggregation

An ER diagram is not capable of representing the relationship between an entity and a relationship which may be required in some scenarios. In those cases, a relationship with its corresponding entities is aggregated into a higher-level entity. Aggregation is an abstraction through which we can represent relationships as higher-level entity sets.

For Example, an Employee working on a project may require some machinery. So, REQUIRE relationship is needed between the relationship WORKS_FOR and entity MACHINERY. Using aggregation, WORKS_FOR relationship with its entities EMPLOYEE and PROJECT is aggregated into a single entity and relationship REQUIRE is created between the aggregated entity and MACHINERY.



Aggregation

Representing Aggregation Via Schema

To represent aggregation, create a schema containing the following things.

- the primary key to the aggregated relationship
- the primary key to the associated entity set
- descriptive attribute, if exists